Document number: P1485R0
Project: Programming Language C++
Audience: Evolution Working group

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2019-02-17

Better keywords for the Coroutines TS

"Our greatest weakness lies in giving up. The most certain way to succeed is always to try just one more time."

— *Thomas A. Edison*

I. Quick Introduction

At the moment Coroutines TS use keywords `co_await`, `co_yield`, and `co_return`. Those keywords were carefully chosen to be "ugly", so that nobody uses them in their code bases and no code break could happen with the Coroutines TS adoption.

This paper revises an approach that allows non "ugly" keywords usage without introducing any breaking change to existing code bases.

II. The idea

Introduce a new context sensitive keyword `async`. All the coroutine definitions must be marked with it. If a function is marked with `async`, then any usage of `yield`, `await` and `return` relate to the coroutines world.

```
struct yield{};
struct await{};

template <class T>
future<int> some_coro() async {
    await something;     // Equivalent to co_await from Coroutines TS
    yield something2;    // Equivalent to co_yield from Coroutines TS
    return something3;   // Equivalent to co_return from Coroutines TS
}

template <class T>
T not_a_coro() {
    await something;     // `something` is an instance of `struct await`
    yield something2;    // `something2` is an instance of `struct yield`
    return something3;   // returning a variable `something3`
}
```

III. Disadvantages

- Users are forced to type `async` on the function definition. That gives at least 1 additional key press for simple cases when only one keyword is used in the coroutine body.
- There are rare border cases when you can not identify what `yield x;` means without looking at the beginning of the function definition
- `co_await`, `co_yield`, and `co_return` are unique. It makes simpler to search them in the Internet.
- Using existing functions or classes with name `yield` while writing new code with coroutines cause problems.

IV. Advantages

- No need in "ugly" `co_await`, `co_yield`, and `co_return`
- Promise type could be partially validated by the compiler without looking into the coroutine body
- `return` in a coroutine does not cause annoying compile time error
- Users may put `async` on declarations to highlight that function is a coroutine
- `return` is just a return. Less differences between functions and coroutines
- `co_await`, `co_yield`, and `co_return` still have a chance to break **existing** code. There could be code bases where they are already used.
- `await` and `yield` are not unique to C++, but still searchable in the Internet. They and the `async` are familiar to people who come from other languages.
- IDE could highlight the `yield` and `await` according to the context, which makes them distinguishable from classes and functions without looking at the beginning of the coroutine.

Coroutines TS	This proposal

`template <class T>`
`T function(string s) {`
`  if (s.empty()) {`
`  return {}; // ill formed`
`  }`
 	
`  co_await query(s);`
`  co_yield s;`
`  co_return {"Done: " + s};`
`}`
	`template <class T>`
	`T function(string s) async {`
	`  if (s.empty()) {`
	`  return {}; // OK`
	`  }`
	
	`  await query(s);`
	`  yield s;`
	`  return {"Done: " + s};`
	`}`

`// 3rd paty code` `struct yield{};` `struct await{};` `template <class T>` `T not_a_coro() {` `  await something;  // `something` is a `struct await`` `  yield something2;  // `something2` is a `struct yield` ` return something3; // returning a variable `something3` `}`	`// 3rd paty code` `struct yield{};` `struct await{};` `template <class T>` `T not_a_coro() { // OK, no `async`, nothing is broken` `  await something;` `  yield something2;` `  return something3;` `}`

`future<int> f(stream str)`
`{`
`  vector<char> buf = ...;`
`  int count = co_await str.read(512, buf);`
`  co_await str.write(512, buf);`
`  co_return count + 11;`
`}`
	`future<int> f(stream str) async`
	`{`
	`  vector<char> buf = ...;`
	`  int count = await str.read(512, buf);`
	`  await str.write(512, buf);`
	`  return count + 11;`
	`}`

`// Is it a coroutine?`
`template <class T>`
`T function(string s);`
	`// This is a coroutine`
	`template <class T>`
	`T function(string s) async ;`

`  yield x{1, 2, 3}; // new variable of type `yield`` `  // ...` `  yield x; // new variable of type `yield`` `  // ...` `  co_yield x;`	`  yield x{1, 2, 3};` `  // ...` `  yield x;` `  // ...` `  yield x;`
`  auto x = []() noexcept -> future { co_await z; };` `  co_await x();`	`  auto x = []() async noexcept -> future { await z; };` `  await x();`

If the new `async` keyword is not acceptable, the following input could change it be used to preview the above table with other keywords (`coro`,`nonlin`,`await`,`gap`):

async

V. History of the problem

Early versions of the coroutines (N3722 for example) were using a special keyword `resumable` to highlight that the function is a coroutine:

```
future<int> f(stream str) resumable
{
  shared_ptr<vector<char>> buf = ...;
  int count = await str.read(512, buf);
  return count + 11;
}
```

That keyword was dropped somewhere around the N4286 while keeping the `await` and `yield`:

```
std::future<void> tcp_reader(int total)
{
  char buf[64 * 1024];
```

```
  auto conn = await Tcp::Connect("127.0.0.1", 1337);
  do
  {
    auto bytesRead = await conn.read(buf, sizeof(buf));
    total -= bytesRead;
  }
  while (total > 0);
}
```

After N4402, the `await` and `yield` were changed to keyword-placeholders [discussion]: `the reason "yield" wasn't used was afraid about breaking code in finance and agriculture`. Later, the "ugly" versions of the keywords were introduced [discussion].

Gor Nishanov explained the `resumable` keyword removal:

```
    1) compiler knows that a function is a coroutine
    2) it forces you to write some kind of tag on a function anyway
```

Gor also noted, that during the keywords discussion in Lexena 2015 he proposed to bring some tag back, but that idea was shouted down at that moment.

Nowadays, 4 years later, people on the reflector and probably outside the WG21 find the solution with `async` like keywords tempting. Because of that and because the "ugly" keywords and the `resumable` keyword did not met, we propose to revise the idea based on the lessons learned on ~4 years of experience with `co_*`.

VI. Using `yield` and `await` functions/classes in coroutines

For an already written code with functions/classes named `yield` or `await` **nothing** gets broken with this paper.

Writing a **new code** with coroutines **and** with functions/classes named `yield` or `await` may require some workarounds.

```
void yield();
struct await{};

template <class T>
future<int> some_coro() async {
    await something;     // co_await, not a class. Compile time error
    yield();             // co_yield, not a function call. Compile time error
}
```

The workarounds for the above code are quite simple, and require either renaming the function and class, or adding a type alias and a function with other name:

```
void yield();
struct await{};

// workarounds
void corn_yield() { yield(); }
using corn_await = await;

template <class T>
future<int> some_coro() async {
    corn_await something;     // OK
    corn_yield();             // OK
}
```

Note that such workarounds do **not** prevent ADL or templates usage:

```
struct await{};
struct foo{};
struct tst{ void yield() };

void yield(await aw);
void yield(foo f);

// workarounds
template <class T> void adl_yield(T v) { yield(v); }
void adl_yield(tst v) { v.yield(); }

template <class T>
future<int> some_coro(T val) async {
    adl_yield(val);             // OK
}

auto res = some_coro(await{});  // OK
```

VII. Distinguishability

A concern of distinguishability was raised on reflector during the proposal discussion. Here's a few examples on border cases when we see "only a single line of code in function":

Coroutines TS	This proposal
`yield x{1, 2, 3};`	`yield x{1, 2, 3};`
`yield x;`	`yield x;`
`co_yield x;`	`yield x;`
`co_yield (x == 0);`	`yield (x == 0);`
`yield(x == 0);`	`yield(x == 0);`

Note that the above problem only arises if we do not see the beginning of the function. Otherwise it's obvious:

Coroutines TS	This proposal
`type foo() {` `  yield x{1, 2, 3};`	`type foo() {` `  yield x{1, 2, 3};`
`type foo() {` `  yield x;`	`type foo() {` `  yield x;`
`type foo() {` `  co_yield x;`	`type foo() async {` `  yield x;`
`type foo() {` `  co_yield (x == 0);`	`type foo() async {` `  yield (x == 0);`
`type foo() {` `  yield(x == 0);`	`type foo() {` `  yield(x == 0);`

VIII. Wording, relative to N4775

1. Remove [lex] and [lex.key]
2. Change [dcl.fct.def.coroutine]:

   A function is a coroutine if ~~it contains a coroutine return statement (9.6.3.1), an await expression (8.3.8), a yield expression (8.21), or a range-based for (9.5.4) with co_await~~its definition marked with `async`.

3. Change [expr.await]:

   The `co_await` expression appears only in coroutines and is used to suspend evaluation of a coroutine (11.4.4) while awaiting completion of the computation represented by the operand expression.

4. Change [stmt.return.coroutine]:

   9.6.3.1 The co~~-~~routine return statement [stmt.return.coroutine]

   ...

A coroutine returns to its caller or resumer (11.4.4) by the co_return statement or when suspended (8.3.8). ~~A coroutine shall not return to its caller or resumer by a return statement(9.6.3).~~

5. Remove all the co_ prefixes from all the occurrences of `co_await`, `co_yield`, and `co_return`.
6. Add to the [dcl.fct] p1 and p2 respectively:

```
D1 ( parameter-declaration-clause ) cv-qualifier-seq_opt
        ref-qualifier_opt async_opt noexcept-specifier_opt attribute-specifier-seq_opt
        ...
D1 ( parameter-declaration-clause ) cv-qualifier-seq_opt
        ref-qualifier_opt async_opt noexcept-specifier_opt attribute-specifier-seq_opt trailing-return-type
```

7. Add to the [dcl.decl] p5 grammar:

```
parameters-and-qualifiers:
        ( parameter-declaration-clause ) cv-qualifier-seq_opt
                ref-qualifier_opt async_opt noexcept-specifier_opt attribute-specifier-seq_opt
```

IX. Acknowledgements

Many thanks to Gor Nishanov, for exploring the coroutines design space, for TS implementations, for teaching people about the coroutines, and for an insane amount of interesting coroutines related measurements/talks/presentations.

Thanks to Ville Voutilainen and Bjarne Stroustrup for pointing me to the previous discussions of the problem and to some concerns.